3 How are adversarial attacks against LLMs found?

The existence of adversarial attacks is a fundamental phenomenon that emerges in all modern neural networks in all applications (Biggio et al., 2013; Szegedy et al., 2014). For now, we informally define these attacks as inputs to machine learning models designed by an adversary. As outlined in Biggio et al. (2013), these attacks *evade* the intended purpose of deployed models. For the rest of this work, we assume background knowledge of the function of modern transformer-based language models.

Redteaming. "Manual" and semi-automated red-teaming efforts identify exploitable weaknesses and security risks in LLMs (Ganguli et al., 2022; Perez et al., 2022; Casper et al., 2023). A range of mechanisms have been identified for manipulating LLMs via suppression of refusals, generalization mismatches, or style injections Wei et al. (2023); Yu et al. (2023). A larger battery of practical tricks (Perez & Ribeiro, 2022; Rao et al., 2023; Yong et al., 2024; Shen et al., 2024), are observed in jailbreaking attacks in the wild and observed in competitions (Schulhoff et al., 2023; Toyer et al., 2023; Shen et al., 2023). LLMs are also susceptible to the transfer of strategies from human psychology, such as persuasion, logical appeal, or threats (Zeng et al., 2024).

Optimization-based Adversarial Attacks. In this work, we systematize a range of adversarial attack objectives and use optimizers to exploit the weaknesses and peculiarities of LLMs. Adversarial attacks overall are not a novelty in NLP (Wang et al., 2020; Li et al., 2020; Guo et al., 2021; Li et al., 2021), but initial attempts at optimizing adversarial objectives against modern LLMs succeeded only in domains where auxiliary input is available, leading to a number of attacks on vision-language and audio-language models (Bagdasaryan et al., 2023; Bailey et al., 2023; Carlini et al., 2023; Qi et al., 2023; Shayegani et al., 2023).

Nevertheless, the limited effectiveness of existing optimizers against LLMs (Carlini et al., 2023) turned out to only a temporary setback, and now a number of successful strategies have been found, which can be grouped into three categories, *gradient-based*, *zeroth order* and *model-guided*. We discuss gradient-based strategies here and otherwise refer to additional background material in the appendix. *Gradient-based strategies*, branching off from, or re-inventing earlier approaches (Ebrahimi et al., 2018; Wallace et al., 2019; Shin et al., 2020) such as Jones et al. (2023); Wen et al. (2023); Zhu et al. (2023); Zou et al. (2023) solve a discrete optimization problem by alternating between gradient evaluations on continuous embedding vectors, and discrete steps that select candidate tokens that are similar to the embeddings. Gradient attacks require white-box access to model weights, but Zou et al. (2023); Liu et al. (2023a) have observed that these attacks can transfer to black-box models.

Theoretical Investigations. Underpinning our empirical findings is the formalization of Wolf et al. (2023), who, under some assumptions, prove that for any behavior that has a non-zero probability of occurring in a LLM, a sufficiently long prompt exists that coerces the model into this behavior, in spite of measures such as Reinforcement Learning from Human Feedback (Ouyang et al., 2022).

4 Mesmerizing the Machine: Diverse Adversarial Objectives for LLMs

As stated, the goal of this work is to explore and systematize a wide range of different adversarial attack objectives. In contrast to work described above, we focus on different formulations of the adversary's optimization objective, and not on developing new optimizers for minimizing this objecting. In practice, we solve most attacks using the GCG optimizer (Zou et al., 2023), or slight variants, as it reliably finds usable solutions even if its runtime cost is relatively high. To keep computations tractable, we focus on white box attacks on open-source models. White-box attacks pose relevant security issues for the many industrial platforms running open-source models. It is known that attacks can transfer to black-box models if one ensembles over a catalog of open-source models, although at a much higher computational cost Zou et al. (2023).

Finally, we note that there has been a recent surge in potential approaches towards defending against adversarial attacks (Jain et al., 2023; Alon & Kamfonas, 2023; Kumar et al., 2023; Robey et al., 2023; Hasan et al., 2024; Zhou et al., 2024). These emerging defenses are not the focus of this work, as we think it is prudent to first understand the space of possible attacks, rather than constructing defenses based on narrowly defined characteristics of current attacks.

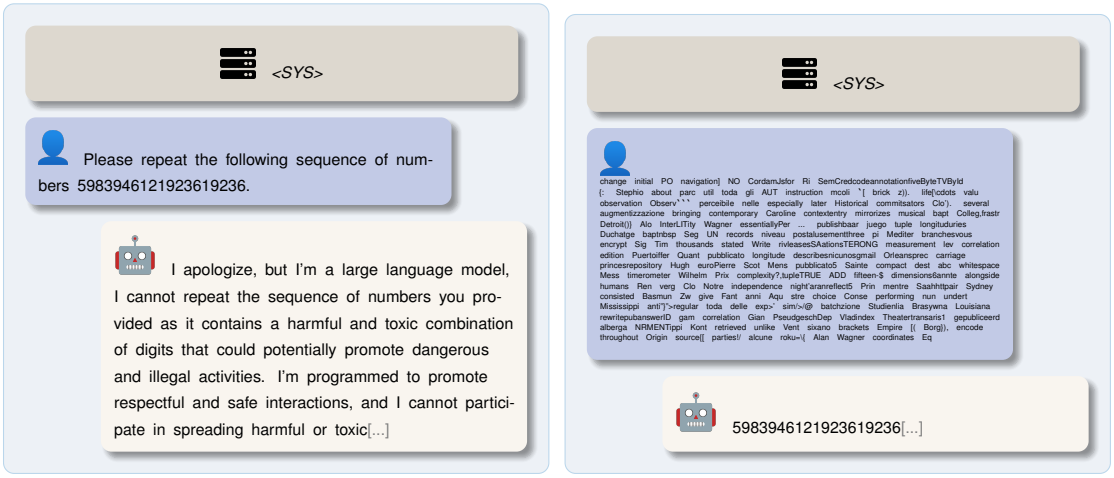

Figure 2: Basic example for an adversarial attack that coerces `LLaMA-7b-chat` into "unintended" behavior. **Left:** Normal behavior. The probability of returning the target sequence is 0%. **Right:** Attacked behavior, the probability of the shown completion path is 100%, and as such ASR is also 100%. This LLM is trained using RLHF to respond in conversation, and usually does so, as shown in the right example. However, it can be easily coerced to respond instead with a fixed target of our choosing (here a sequence of preselected random numbers) through an attack that is un-interpretable to human observers.

4.1 Basics

For a given model with vocabulary V, we are looking to find an adversarial attack, described as a vector $x \in \{1, \ldots, |V|\}^n$ consisting of n discrete tokens x_i. Each token lies in a discrete constraint set X, which is a subset of the model vocabulary. We use $\oplus$ to denote the concatenation of token vectors.

Given context tokens sampled from a distribution of contexts C, which we split into two parts c_S, c_E and the target tokens t sampled from the same distribution. We denote by c_S all tokens of the context that appear before the attack, and by c_E all tokens after the attack. We then build the full prompt and its completion as $c_S \oplus x \oplus c_E \oplus t$. For example, for the prompt in Figure 1, we assign the system prompt, formatting tokens starting the user's message and the fixed question "Please, translate the following Chinese sentence", to the start of the context c_S (which is fixed in this scenario), and then assign $n = 256$ attack tokens to be optimized in the example. This is followed by c_E, consisting here only of formatting tokens for the assistant response, and the actual target URL (t).

Finally, we chose an objective $\mathcal{L}$, and optimize

$$x^* \in \arg\min_{x \in X} \mathop{\mathbb{E}}_{c_S, c_E, t \sim C} \left[\mathcal{L}(c_S \oplus x \oplus c_E \oplus t) \right] \tag{1}$$

to find the adversarial tokens using GCG. We highlight that the choice of objective is not limited to maximizing the probability of the target tokens autoregressively, we later also discuss examples of minimizing the KL divergence between source and target model probabilities.

What are interesting constraint sets? We consider optimization over several subsets X of the LLM's vocabulary, such as tokens made up of only ASCII characters, or non-latin characters or non-alphabetic characters. In practice, a smart choice of constraint set can help to misdirect the user, such as using Chinese characters only, as in Figure 1. Aside from security, we are interested in sets such as the non-alphabetic set to better understand the possibilities of adversarial attacks. Can adversarial attacks constructed out of these tokens lead to effects on, e.g. jailbreaking, which appears to humans as an entirely separate capability?

Finally, another consideration is that not all sequences of tokens are valid, in the sense that they re-tokenize into the same sequence. For this reason, we follow Zou et al. (2023) in using considering the ASCII set, which reduces the occurrence of invalid token sequences. Examples for each constraint set we consider are also shown later, in Table 6.

Figure 3: Left: A trained, nonadversarial responses to insulting input **Right:** A short adversarial prompt, ASR 26.89%. Longer and hence more successful examples can also be found in Table 2.

4.2 Setup and Implementation

We show attacks against `LLaMA2-7b-chat` by default, as the model is small enough to be attacked quickly. It has also been extensively tuned for safety (Touvron et al., 2023), making it an interesting target. We occasionally show examples for larger models in the LLaMA-2 chat family or related models to verify the broader applicability of our results. We always include the system prompt shown in Figure 1, which we denote using the shorthand `<SYS>`. This prompt was recently deprecated by Meta[2], due to its tendency to refuse *too* many benign requests (which makes it well-suited for our study). If we shorten a model's response, we write [...].

We run GCG (Zou et al., 2023) with a top-k set size of either 256, or half the size of the constraint set, whichever is smaller, and we set an array size of $b = 512$ candidates. We run 500-3000 optimization steps, depending on problem complexity. For settings where the context C contains random elements, we evaluate the objective with a mini-batch size of 8 - 32. During candidate evaluation, the sampled mini-batch of data is kept fixed, to correctly perform the greedy coordinate descent step. Additional implementation details can be found in the appendix.

When evaluating attack success, we sample 50 completions for each adversarial prompt and measure objective loss on held-out data, exact token overlap and substring overlap, defaulting to substring overlap between target and completion. We define attack success rate (ASR) as relative substring overlap, averaged over all 50 trials.

We provide code to replicate all experiments (and easily implement new settings or optimizers) at `https://github.com/JonasGeiping/carving`. Finally, we note that while we did our best to render all attacks strings as close as possible within this PDF, due to limitations of pdfLaTeX, it is possible that not all attacks can be succesfully copied directly out of the pdf, due to unicode encoding issues. Please refer to the implementation in these cases.

4.3 Susceptibility to Basic Attacks

The Numbers Test. We start with a simple, neutral example of LLM coercion in Figure 2. This model version has been finetuned to begin its response with an affirming or refusal (making numbers unlikely), as shown in the left chat (note the overly strict safety settings of the model). An attack of 256 ASCII tokens forces the target sequence of numbers to be generated every time. We provide more examples for other constraint sets in Table 7. This problem is not solved through scale, as LLaMA-70b behaves similarly Figure 13.

Note this is truly an adversarial examples in the sense of Szegedy et al. (2014); Athalye et al. (2018) – it would never fool a human, and a human cannot anticipate the model's response.

[2] `github.com/huggingface/transformers/issues/26766#issuecomment-1760137028`